TP 5 :

Dr. Hamouichou Khawla

Résumé sur les Tableaux :

1. Définition :

Un **tableau** (array) est une structure de données permettant de stocker plusieurs valeurs **du même type** dans un espace mémoire contigu.
Chaque élément est accessible par son **indice**.

2. Types de tableaux :

✔ Tableau 1D (une dimension)

Représente une **liste** d'éléments.

Exemple :

`A[5]` = tableau de 5 éléments.

Accès :

- `A[0]` → premier élément
- `A[n-1]` → dernier élément

✔ Tableau 2D (deux dimensions)

Représente une **matrice** de lignes et colonnes.

Exemple :

`M[3][4]` = matrice de 3 lignes et 4 colonnes.

Accès :

- `M[i][j]` → élément de la ligne *i* et colonne *j*

3. Déclaration :

En C

```c
int A[10];
int M[5][5];
```

4. Saisie et affichage :

Tableau 1D

```c
for(i=0; i<n; i++) scanf("%d", &A[i]);
for(i=0; i<n; i++) printf("%d ", A[i]);
```

Tableau 2D

```c
for(i=0; i<L; i++)
    for(j=0; j<C; j++)
        scanf("%d", &M[i][j]);
```

5. Opérations courantes :

✅ a) Recherche d'un élément

On parcourt le tableau jusqu'à trouver la valeur.

✅ b) Maximum / Minimum

On compare chaque élément au max/min courant.

✅ c) Transfert 2D → 1D

On copie ligne par ligne les données d'une matrice vers un tableau simple.

✅ d) Comptage, somme, moyenne

On additionne éléments et divise par le nombre total.

🔷 8. Exemple global (tableau 1D)

```c
int A[5] = {2, 8, 3, 7, 1};
```

- Max = 8
- Min = 1
- Position de 7 = 3
- Somme = 21
- Moyenne = 4.2

Concept	Description
Tableau	Structure pour stocker plusieurs valeurs du même type
1D	Liste linéaire
2D	Matrice (lignes × colonnes)
Accès	Par indice (A[i], M[i][j])
Opérations	Recherche, max/min, somme, transfert 2D→1D
Limite	Taille fixe

👉 C'est quoi V[L*C] ?

Dans un exercice, on a souvent un tableau **2D** (à deux dimensions) comme :

```
M[L][C]
```

- **L** = nombre de lignes
- **C** = nombre de colonnes

Donc le tableau contient au total :

```
L * C éléments
```

Si on veut transformer ce tableau **2D** en tableau **1D** (une seule ligne), on crée un tableau :

```
V[L*C]
```

✅ Signification

`V[L*C]` veut dire :
➡️ Un tableau **V** qui contient **L × C cases**, c'est-à-dire exactement le même nombre d'éléments que le tableau 2D.

Exemple très simple

Si

- L = 3
- C = 4

alors :

```markdown
V[L*C] = V[3*4] = V[12]
```

Donc V est un tableau **à 12 éléments.**

Exercice 1 — Transférer un tableau 2D M [L] [C] vers un tableau 1D V [L*C] :

Idée : parcourir les lignes puis les colonnes, et remplir V en ajoutant les éléments dans l'ordre ligne-par-ligne.

```c
int main() {
    int L, C;
    int M[10][10];
    int V[100];
    int i, j, idx;

    // saisie dimensions
    printf("Entrer le nombre de lignes (<=10): ");
    scanf("%d", &L);
    printf("Entrer le nombre de colonnes (<=10): ");
    scanf("%d", &C);

    if (L <= 0 || L > 10 || C <= 0 || C > 10) {
        printf("Dimensions invalides.\n");
        return 1;
    }
```

```c
    // saisie du tableau 2D
    printf("Saisir les %d*%d elements du tableau M:\n", L, C);
    for (i = 0; i < L; i++) {
        for (j = 0; j < C; j++) {
            printf("M[%d][%d] = ", i, j);
            scanf("%d", &M[i][j]);
        }
    }

    // affichage du 2D
    printf("\nTableau M :\n");
    for (i = 0; i < L; i++) {
        for (j = 0; j < C; j++) {
            printf("%d ", M[i][j]);
        }
        printf("\n");
    }
```

```c
    // transfert vers V
    idx = 0;
    for (i = 0; i < L; i++) {
        for (j = 0; j < C; j++) {
            V[idx++] = M[i][j];
        }
    }

    // affichage du 1D
    printf("\nTableau V (1D) :\n");
    for (i = 0; i < L*C; i++) {
        printf("%d ", V[i]);
    }
    printf("\n");
    return 0;
}
```

Exercice 2 — Trouver le maximum et le minimum (valeur + position)

Idée : parcourir le tableau A, garder la valeur max et min rencontrées et leurs premières positions (index du premier élément rencontré). Si on veut les positions sous la forme indice (0-based) ou i, j si 2D — ici A est 1D d'après l'énoncé.

```c
#include <stdio.h>

int main() {
    int n, i;
    printf("Entrer la taille du tableau: ");
    scanf("%d", &n);
    if (n <= 0) {
        printf("Taille invalide.\n");
        return 1;
    }
```

```c
    int A[n];
    for (i = 0; i < n; i++) {
        printf("A[%d] = ", i);
        scanf("%d", &A[i]);
    }

    // initialisation
    int max = A[0], min = A[0];
    int posMax = 0, posMin = 0;
```

```c
    // initialisation
    int max = A[0], min = A[0];
    int posMax = 0, posMin = 0;

    for (i = 1; i < n; i++) {
        if (A[i] > max) {
            max = A[i];
            posMax = i; // conserver première position du max
        }
        if (A[i] < min) {
            min = A[i];
            posMin = i; // conserver première position du min
        }
    }
```

```c
    printf("\nTableau saisi :\n");
    for (i = 0; i < n; i++) printf("%d ", A[i]);
    printf("\n");

    printf("Maximum = %d a la position %d\n", max, posMax);
    printf("Minimum = %d a la position %d\n", min, posMin);

    return 0;
}
```

Exercice 3 — Recherche d'une valeur VAL dans un tableau A

Idée : parcourir A et si on trouve VAL, mémoriser l'indice dans POS et arrêter (ou continuer si on souhaite trouver toutes les occurrences). Suivre la consigne : POS = -1 tant que non trouvée ; garder la première position.

```c
#include <stdio.h>

int main() {
    int n, i;
    printf("Entrer la taille du tableau: ");
    scanf("%d", &n);
    if (n <= 0) {
        printf("Taille invalide.\n");
        return 1;
    }
```

```c
  int A[n];
  for (i = 0; i < n; i++) {
      printf("A[%d] = ", i);
      scanf("%d", &A[i]);
  }

  int VAL;
  printf("Entrer la valeur a rechercher VAL = ");
  scanf("%d", &VAL);
```

```c
  int POS = -1;
  for (i = 0; i < n; i++) {
      if (A[i] == VAL) {
          POS = i;
          break; // on garde la première position rencontrée
      }
  }
```

```c
    if (POS != -1)
        printf("La valeur %d se trouve a la position %d\n", VAL, POS);
    else
        printf("La valeur %d n'est pas dans le tableau.\n", VAL);

    return 0;
}
```